

دليل Frontend لواجهات Authentication و OTP

الهدف من هذا الملف

هذا الدليل موجه لفريق الـ frontend حتى يطبق الـ API الجديدة الخاصة بـ:

- register
- login
- verify-otp
- resend-verification-otp
- request-otp
- refresh
- logout
- forgot-password
- reset-password

ويشرح بالتفصيل كيف تغيّر الـ flow، ما الذي يجب إرساله في كل request، ما الذي سيعود في كل response، وكيف يجب أن يتصرف التطبيق في كل حالة.

هذا الدليل يركّز على email/phone + OTP authentication flow .
توجد أيضاً routes خاصة بـ Google auth داخل api/auth/ , لكنها ليست محور هذا التعديل ولذلك ذكرتها في جدول الراوترات فقط كمرجع.

TL;DR

ما الذي تغيّر؟

السلوك القديم كان تقريباً:

1. المستخدم يعمل register .
2. النظام ينشئ الحساب.
3. النظام يرسل OTP.
4. النظام كان يعطي access_token و refresh_token مباشرة.

السلوك الجديد أصبح:

1. المستخدم يعمل register .
2. النظام ينشئ الحساب ويرسل OTP .
3. لا يعطي أي tokens في register .
4. إذا كان الحساب غير مفعل، فإن login يعيد 403 verification challenge وليس tokens .
5. المستخدم يجب أن يكمل verify-otp .
6. فقط بعد verify-otp الناجح يتم إصدار access_token و refresh_token .

القاعدة الذهبية للـ frontend

- لا تخزن أي tokens بعد register .
- لا تفترض أن login دائماً يرجع 200 .
- إذا رجع 403 ومعه requires_otp_verification=true ، انقل المستخدم إلى شاشة OTP .
- لا تعتمد على نص message وحده لاتخاذ القرار.
- اعتمد على status code + flags مثل:
 - requires_otp_verification
 - registration_method
 - is_account_verified
 - masked_destination

Base URL و Headers

كل الأمثلة هنا تفترض أن الـ base URL هو:

api/{{base_url}}

Headers العامة

لجميع طلبات JSON:

Accept: application/json

Content-Type: application/json

لجميع الـ protected routes:

Authorization: Bearer YOUR_ACCESS_TOKEN

القيم الأساسية التي يجب أن يفهمها الـ frontend

1. registration_method

هذه القيمة تحدد هل الحساب مبني على:

- email
- phone

وهي الآن جزء أساسي من معظم auth requests.

2. verification_channel

حالياً تساوي نفس قيمة registration_method :

- email
- phone

لكن يفضل التعامل معها كحقل display/use-case خاص بقناة التحقق.

3. masked_destination

هذا الحقل يوضح للمستخدم أين تم إرسال الـ OTP، لكن بشكل masked.

أمثلة:

- ah*****@example.com
- 7890*****

4. flags الخاصة بالتفعيل

هذه الحقول تظهر كثيراً في responses المتعلقة بالتفعيل:

```
{
  "registration_method": "email",
  "verification_channel": "email",
  "masked_destination": "ah*****@example.com",
  "email_verified": false,
  "phone_verified": false,
  "is_account_verified": false,
  "requires_otp_verification": true
}
```

5. otp في ال response

في بعض البيانات التطويرية قد يظهر حقل otp داخل ال response إذا كان:

```
APP_DEBUG=true
```

مثال:

```
}
"otp": "123456"
{
```

مهم جداً: ال frontend يجب ألا يعتمد على هذا الحقل نهائياً في production. هذا الحقل فقط للتجربة المحلية و debugging.

شكل object user في auth responses

ال backend يعيد user عبر UserResource بالشكل التالي تقريباً:

```

{
  "id": 15,
  "first_name": "Ahmad",
  "last_name": "Ali",
  "full_name": "Ahmad Ali",
  "email": "ahmad@example.com",
  "phone": null,
  "registration_method": "email",
  "address": null,
  "city": null,
  "avatar_url": null,
  "profile_image_url": null,
  "is_active": true,
  "email_verified_at": null,
  "phone_verified_at": null,
  "is_account_verified": false,
  "requires_otp_verification": true,
  "created_at": "2026-05-02T10:00:00.000000Z",
  "updated_at": "2026-05-02T10:00:00.000000Z"
}

```

ملاحظات مهمة

- إذا كان `registration_method=email` فغالباً `phone` سيكون `null`.
- إذا كان `registration_method=phone` فغالباً `email` سيكون `null`.
- لا تفترض أن كلا الحقلين موجودان دائماً.

خريطة الراوترات المتعلقة بـ Authentication

أهم Status Codes	يحتاج Token؟	الهدف	Route	Method
201 , 422	لا	إنشاء حساب جديد بدون tokens	api/auth/register/	POST
200 , 401 , 422 , 403	لا	تسجيل الدخول أو إرجاع verification challenge	api/auth/login/	POST

أهم Status Codes	يحتاج Token؟	الهدف	Route	Method
422 , 200	لا	التحقق من OTP وإصدار tokens	api/auth/verify-otp/	POST
422 , 200	لا	email alias قديم لـ verification	api/auth/verify-email-otp/	POST
, 400 , 200 422	لا	إعادة إرسال OTP لحساب غير مفعل	api/auth/resend-verification-otp/	POST
, 422 , 200 404	لا	طلب OTP عام بالبريد أو الهاتف	api/auth/request-otp/	POST
, 401 , 200 422 , 403	لا	إنشاء access token جديد من refresh token	api/auth/refresh/	POST
401 , 200	نعم	تسجيل الخروج وإلغاء التوكينات	api/auth/logout/	POST
, 422 , 200 404	لا	إرسال OTP لإعادة تعيين كلمة المرور عبر email فقط	api/auth/forgot-password/	POST
, 422 , 200 404	لا	إعادة تعيين كلمة المرور عبر OTP	api/auth/reset-password/	POST
خارج نطاق هذا الدليل	لا	Google auth	api/auth/google/	POST
خارج نطاق هذا الدليل	لا	Google mobile auth	api/auth/google/mobile/	POST
خارج نطاق هذا الدليل	لا	Google web redirect	api/auth/google/redirect/	GET
خارج نطاق هذا الدليل	لا	Google web callback	api/auth/google/callback/	GET

1. POST /api/auth/register

الهدف

إنشاء حساب جديد وإرسال OTP، لكن بدون login مباشر وبدون أي tokens.

متى يستخدمه الـ frontend؟

- عند إنشاء حساب جديد عبر email.
- عند إنشاء حساب جديد عبر phone.

Request: تسجيل عبر email

POST /api/auth/register

```
{
  "first_name": "Ahmad",
  "last_name": "Ali",
  "registration_method": "email",
  "email": "ahmad@example.com",
  "password": "Password@123",
  "password_confirmation": "Password@123"
}
```

Success Response: 201 Created

```
{
  "user": {
    "id": 15,
    "first_name": "Ahmad",
    "last_name": "Ali",
    "full_name": "Ahmad Ali",
    "email": "ahmad@example.com",
    "phone": null,
    "registration_method": "email",
    "address": null,
    "city": null,
    "avatar_url": null,
    "profile_image_url": null,
    "is_active": true,
    "email_verified_at": null,
    "phone_verified_at": null,
    "is_account_verified": false,
    "requires_otp_verification": true,
    "created_at": "2026-05-02T10:00:00.000000Z",
    "updated_at": "2026-05-02T10:00:00.000000Z"
  },
  "message": "Registration successful. Please verify your email using the OTP sent to your email.",
  "registration_method": "email",
  "verification_channel": "email",
  "masked_destination": "ah***@example.com",
  "email_verified": false,
  "phone_verified": false,
  "is_account_verified": false,
  "requires_otp_verification": true
}
```

Request: تسجيل عبر phone

```
{  
  "first_name": "Ahmad",  
  "last_name": "Ali",  
  "registration_method": "phone",  
  "phone": "+491234567890",  
  "password": "Password@123",  
  "password_confirmation": "Password@123"  
}
```

Success Response: 201 Created

```
{
  "user": {
    "id": 16,
    "first_name": "Ahmad",
    "last_name": "Ali",
    "full_name": "Ahmad Ali",
    "email": null,
    "phone": "+491234567890",
    "registration_method": "phone",
    "address": null,
    "city": null,
    "avatar_url": null,
    "profile_image_url": null,
    "is_active": true,
    "email_verified_at": null,
    "phone_verified_at": null,
    "is_account_verified": false,
    "requires_otp_verification": true,
    "created_at": "2026-05-02T10:00:00.000000Z",
    "updated_at": "2026-05-02T10:00:00.000000Z"
  },
  "message": "Registration successful. Please verify your phone number using the OTP sent to you",
  "registration_method": "phone",
  "verification_channel": "phone",
  "masked_destination": "*****7890",
  "email_verified": false,
  "phone_verified": false,
  "is_account_verified": false,
  "requires_otp_verification": true
}
```

Validation Errors: 422 Unprocessable Entity

مثال إذا نسيت registration_method أو password_confirmation :

```

{
  "message": "The registration method field is required. (and 1 more error)",
  "errors": {
    "registration_method": [
      "The registration method field is required."
    ],
    "password": [
      "The password field confirmation does not match."
    ]
  }
}

```

ماذا يجب أن يفعل الـ frontend بعد register ؟

1. لا تخزن أي tokens.
2. انتقل مباشرة إلى شاشة OTP Verification .
3. مرر إلى الشاشة:

 - registration_method
 - phone أو email
 - masked_destination

4. ابدأ countdown لإتاحة resend-verification-otp .

2. POST /api/auth/login

الهدف

تسجيل الدخول بحساب موجود.

السلوك الجديد

- إذا الحساب verified يرجع tokens بشكل طبيعي.
- إذا الحساب unverified يرجع 403 مع verification challenge ويرسل OTP جديد.
- إذا الـ credentials غلط يرجع 401 .
- إذا الحساب inactive يرجع 403 .

email عبر Request: login

```
{  
  "registration_method": "email",  
  "email": "ahmad@example.com",  
  "password": "Password@123"  
}
```

phone عبر Request: login

```
{  
  "registration_method": "phone",  
  "phone": "+491234567890",  
  "password": "Password@123"  
}
```

Success Response للحساب المفعّل: OK 200

```
{
  "user": {
    "id": 15,
    "first_name": "Ahmad",
    "last_name": "Ali",
    "full_name": "Ahmad Ali",
    "email": "ahmad@example.com",
    "phone": null,
    "registration_method": "email",
    "address": null,
    "city": null,
    "avatar_url": null,
    "profile_image_url": null,
    "is_active": true,
    "email_verified_at": "2026-05-02T10:05:00.000000Z",
    "phone_verified_at": null,
    "is_account_verified": true,
    "requires_otp_verification": false,
    "created_at": "2026-05-02T10:00:00.000000Z",
    "updated_at": "2026-05-02T10:05:00.000000Z"
  },
  "access_token": "1|long_access_token_here",
  "access_expires_at": "2026-05-02T12:00:00.000000Z",
  "refresh_token": "plain_refresh_token_here",
  "refresh_expires_at": "2026-05-09T10:05:00.000000Z",
  "token_type": "bearer",
  "registration_method": "email",
  "verification_channel": "email",
  "masked_destination": "ah***@example.com",
  "email_verified": true,
  "phone_verified": false,
  "is_account_verified": true,
  "requires_otp_verification": false
}
```

Response للحساب غير المفعل: 403 Forbidden

```
{
  "user": {
    "id": 15,
    "first_name": "Ahmad",
    "last_name": "Ali",
    "full_name": "Ahmad Ali",
    "email": "ahmad@example.com",
    "phone": null,
    "registration_method": "email",
    "address": null,
    "city": null,
    "avatar_url": null,
    "profile_image_url": null,
    "is_active": true,
    "email_verified_at": null,
    "phone_verified_at": null,
    "is_account_verified": false,
    "requires_otp_verification": true,
    "created_at": "2026-05-02T10:00:00.000000Z",
    "updated_at": "2026-05-02T10:00:00.000000Z"
  },
  "message": "Your account is not verified. A new OTP has been sent to your registered contact.",
  "registration_method": "email",
  "verification_channel": "email",
  "masked_destination": "ah***@example.com",
  "email_verified": false,
  "phone_verified": false,
  "is_account_verified": false,
  "requires_otp_verification": true
}
```

Response للـ credentials الخاطئة: 401 Unauthorized

```
{
  "message": "Invalid credentials"
}
```

Response للحساب غير النشط: 403 Forbidden

```
{  
  "message": "This account is inactive."  
}
```

ماذا يجب أن يفعل الـ frontend بعد login ؟

إذا كانت النتيجة 200

1. خزن access_token .
2. خزن refresh_token .
3. خزن user إن كنت تحتاجه في session store .
4. انتقل إلى الشاشة الرئيسية.

إذا كانت النتيجة 403 ومعها requires_otp_verification=true

1. لا تعتبر العملية login ناجحة بعد.
2. لا تخزن أي tokens .
3. انتقل إلى شاشة OTP .
4. استخدم registration_method و masked_destination لعرض الرسالة للمستخدم.

إذا كانت النتيجة 401

1. اعرض رسالة Invalid credentials .
2. ابقَ في شاشة login .

3. POST /api/auth/verify-otp

الهدف

هذا هو الـ endpoint الأساسي الذي يُكمل activation ويصدر tokens .

متى يستخدم؟

- بعد register
- بعد login إذا رجع verification challenge

Request: verify email OTP

```
{  
  "registration_method": "email",  
  "email": "ahmad@example.com",  
  "otp": "123456"  
}
```

Request: verify phone OTP

```
{  
  "registration_method": "phone",  
  "phone": "+491234567890",  
  "otp": "123456"  
}
```

Success Response: 200 OK

```
{
  "message": "Email verified successfully.",
  "user": {
    "id": 15,
    "first_name": "Ahmad",
    "last_name": "Ali",
    "full_name": "Ahmad Ali",
    "email": "ahmad@example.com",
    "phone": null,
    "registration_method": "email",
    "address": null,
    "city": null,
    "avatar_url": null,
    "profile_image_url": null,
    "is_active": true,
    "email_verified_at": "2026-05-02T10:05:00.000000Z",
    "phone_verified_at": null,
    "is_account_verified": true,
    "requires_otp_verification": false,
    "created_at": "2026-05-02T10:00:00.000000Z",
    "updated_at": "2026-05-02T10:05:00.000000Z"
  },
  "access_token": "1|long_access_token_here",
  "access_expires_at": "2026-05-02T12:00:00.000000Z",
  "refresh_token": "plain_refresh_token_here",
  "refresh_expires_at": "2026-05-09T10:05:00.000000Z",
  "token_type": "bearer",
  "registration_method": "email",
  "verification_channel": "email",
  "masked_destination": "ah***@example.com",
  "email_verified": true,
  "phone_verified": false,
  "is_account_verified": true,
  "requires_otp_verification": false
}
```

phone verification عند Success Response: 200 OK

```
{
  "message": "Phone number verified successfully.",
  "user": {
    "id": 16,
    "first_name": "Ahmad",
    "last_name": "Ali",
    "full_name": "Ahmad Ali",
    "email": null,
    "phone": "+491234567890",
    "registration_method": "phone",
    "address": null,
    "city": null,
    "avatar_url": null,
    "profile_image_url": null,
    "is_active": true,
    "email_verified_at": null,
    "phone_verified_at": "2026-05-02T10:05:00.000000Z",
    "is_account_verified": true,
    "requires_otp_verification": false,
    "created_at": "2026-05-02T10:00:00.000000Z",
    "updated_at": "2026-05-02T10:05:00.000000Z"
  },
  "access_token": "1|long_access_token_here",
  "access_expires_at": "2026-05-02T12:00:00.000000Z",
  "refresh_token": "plain_refresh_token_here",
  "refresh_expires_at": "2026-05-09T10:05:00.000000Z",
  "token_type": "bearer",
  "registration_method": "phone",
  "verification_channel": "phone",
  "masked_destination": "*****7890",
  "email_verified": false,
  "phone_verified": true,
  "is_account_verified": true,
  "requires_otp_verification": false
}
```

Invalid OTP: 422 Unprocessable Entity

```
{  
  "error": "Invalid or expired OTP"  
}
```

ماذا يجب أن يفعل الـ frontend بعد verify-otp ؟

1. خزّن access_token و refresh_token .
2. خزّن user .
3. أغلق شاشة OTP.
4. انتقل إلى الشاشة الرئيسية أو onboarding التالية.

4. POST /api/auth/verify-email-otp

الهدف

هذا route موجود للتوافق الخلفي backward compatibility مع integrations أقدم.

التوصية للـ frontend الجديد

استخدم دائماً:

POST /api/auth/verify-otp

مع:

```
{  
  "registration_method": "email"  
}
```

متى يمكن استخدام verify-email-otp ؟

إذا كان لديكم كود frontend قديم مبني عليه ولا تريدون تغييره حالياً.

Request Example

```
{  
  "email": "ahmad@example.com",  
  "otp": "123456"  
}
```

Success Response

هو نفس response الخاص بـ verify-otp عند email verification.

5. POST /api/auth/resend-verification-otp

الهدف

إعادة إرسال OTP لحساب موجود لكنه غير مفعل.

متى يستخدم؟

- من شاشة OTP إذا انتهت المهلة أو طلب المستخدم إعادة الإرسال.

Request: email account

```
{  
  "registration_method": "email",  
  "email": "ahmad@example.com"  
}
```

Request: phone account

```
{
  "registration_method": "phone",
  "phone": "+491234567890"
}
```

Success Response: 200 OK

```
{
  "message": "OTP sent to your email.",
  "registration_method": "email",
  "verification_channel": "email",
  "masked_destination": "ah***@example.com",
  "email_verified": false,
  "phone_verified": false,
  "is_account_verified": false,
  "requires_otp_verification": true
}
```

وعند phone يصبح message غالباً:

```
{
  "message": "OTP sent to your phone."
}
```

إذا كان الحساب مفعلاً أصلاً: Bad Request 400

```
{
  "message": "Account already verified."
}
```

Validation/Existence Errors

- إذا لم ترسل email أو phone المناسبين سيعود 422 .

- إذا لم يوجد المستخدم قد تعود 422 أو 404 حسب نقطة الفشل.

ماذا يجب أن يفعل الـ frontend؟

1. اعرض toast مثل OTP resent successfully .
2. أعد تشغيل countdown .
3. لا تغير screen flow .
4. لا تتوقع tokens من هذا endpoint .

6. POST /api/auth/request-otp

الهدف

طلب OTP بشكل عام بواسطة email أو phone .

متى يستخدم؟

- لإرسال OTP بدون login .
- لإعادة طلب OTP في بعض flows العامة.
- يمكن استخدامه أيضاً ضمن password reset flow خاصة للهاتف.

الفرق بينه وبين resend-verification-otp

: request-otp

- endpoint عام
- لا يفحص أن الحساب غير verified قبل الإرسال

: resend-verification-otp

- endpoint مخصص لـ account activation
- يعيد 400 إذا كان الحساب verified بالفعل

Request: email

```
{
  "registration_method": "email",
  "email": "ahmad@example.com"
}
```

Request: phone

```
{
  "registration_method": "phone",
  "phone": "+491234567890"
}
```

Success Response: 200 OK

```
{
  "message": "OTP sent successfully.",
  "registration_method": "phone",
  "verification_channel": "phone",
  "masked_destination": "*****7890",
  "email_verified": false,
  "phone_verified": false,
  "is_account_verified": false,
  "requires_otp_verification": true
}
```

مهم

- هذا endpoint لا يفعل الحساب.
- هذا endpoint لا يعيد tokens.
- هذا endpoint فقط يرسل OTP.

POST /api/auth/refresh .7

الهدف

الحصول على access_token جديد باستخدام refresh_token .

Request

```
{  
  "refresh_token": "plain_refresh_token_here"  
}
```

Success Response: 200 OK

```
{  
  "access_token": "1|new_access_token_here",  
  "access_expires_at": "2026-05-02T12:30:00.000000Z",  
  "requires_otp_verification": false  
}
```

إذا كان الحساب غير مفعّل: Forbidden 403

```
{
  "user": {
    "id": 15,
    "first_name": "Ahmad",
    "last_name": "Ali",
    "full_name": "Ahmad Ali",
    "email": "ahmad@example.com",
    "phone": null,
    "registration_method": "email",
    "address": null,
    "city": null,
    "avatar_url": null,
    "profile_image_url": null,
    "is_active": true,
    "email_verified_at": null,
    "phone_verified_at": null,
    "is_account_verified": false,
    "requires_otp_verification": true,
    "created_at": "2026-05-02T10:00:00.000000Z",
    "updated_at": "2026-05-02T10:00:00.000000Z"
  },
  "message": "Your account is not verified. A new OTP has been sent to your registered contact.",
  "registration_method": "email",
  "verification_channel": "email",
  "masked_destination": "ah***@example.com",
  "email_verified": false,
  "phone_verified": false,
  "is_account_verified": false,
  "requires_otp_verification": true
}
```

إذا كان refresh token غير صالح: Unauthorized 401

```
{
  "message": "Invalid or expired refresh token"
}
```

ملاحظات مهمة للـ frontend

- refresh يعيد access_token جديد فقط.
- لا يعيد refresh_token جديد.
- احتفظ بالـ refresh_token الحالي حتى تنتهي صلاحيته أو يحصل logout.
- إذا رجع 403 ومعه requires_otp_verification=true ، انقل المستخدم إلى شاشة OTP.

8. POST /api/auth/logout

الهدف

تسجيل خروج المستخدم.

Request

POST /api/auth/logout

Authorization: Bearer YOUR_ACCESS_TOKEN

Success Response: 200 OK

```
{  
  "message": "Logged out"  
}
```

ملاحظة مهمة

الـ backend يلغي:

- access tokens
- refresh tokens

لذلك بعد logout يجب على الـ frontend حذف أي session محلية بالكامل.

Password Reset Flow .9

يوجد هنا مساران مهمان:

1. forgot-password

2. reset-password

ملاحظة مهمة جداً

forgot-password الحالي هو endpoint قديم نسيباً ويدعم email فقط.

أما reset-password الحالي فيدعم:

- email
- phone

لكن يجب إرسال registration_method أو type حتى يعرف الـ backend القناة.

9.1 POST /api/auth/forgot-password

الهدف

إرسال OTP لإعادة تعيين كلمة المرور عبر email فقط.

Request

```
{  
  "email": "ahmad@example.com"  
}
```

Success Response: 200 OK

```
{  
  "message": "OTP sent to email"  
}
```

- هذا endpoint لا يعيد verification payload.
- هذا endpoint لا يعيد tokens.
- في local/dev قد يظهر otp إذا كان APP_DEBUG=true.

9.2 POST /api/auth/reset-password

الهدف

إعادة تعيين كلمة المرور بعد إدخال OTP صحيح.

Request: reset via email

```
{
  "registration_method": "email",
  "email": "ahmad@example.com",
  "otp": "123456",
  "password": "NewPassword@123",
  "password_confirmation": "NewPassword@123"
}
```

Request: reset via phone

```
{
  "registration_method": "phone",
  "phone": "+491234567890",
  "otp": "123456",
  "password": "NewPassword@123",
  "password_confirmation": "NewPassword@123"
}
```

Success Response: 200 OK

```
{  
  "message": "Password reset successful"  
}
```

Invalid OTP: 422 Unprocessable Entity

```
{  
  "error": "Invalid or expired OTP"  
}
```

كيف يبني الـ frontend flow الصحيح؟

Password reset via email

1. اطلب POST /api/auth/forgot-password مع email .
2. افتح شاشة إدخال OTP + password الجديدة.
3. أرسل POST /api/auth/reset-password مع:
 - registration_method=email
 - email
 - otp
 - password
 - password_confirmation

Password reset via phone

لأنه لا يوجد endpoint مخصص باسم forgot-password-phone حالياً، استخدم:

1. POST /api/auth/request-otp
2. ثم POST /api/auth/reset-password

مع:

```

}
, "registration_method": "phone"
"phone": "+491234567890"
{

```

ثم:

```

}
, "registration_method": "phone"
, "phone": "+491234567890"
, "otp": "123456"
, "password": "NewPassword@123"
"password_confirmation": "NewPassword@123"
{

```

10. سلوك الـ protected routes بعد هذا التعديل

بعض الـ routes المحمية للمستخدم customer أصبحت تتطلب:

- auth:sanctum
- وأن يكون الحساب verified

إذا استخدم access token لحساب customer غير مفعّل، سيعود 403 مثل هذا:

```

}
Your account is not verified. Please verify it using the OTP sent to your registered contact"
, "registration_method": "email"
, "email_verified": false"
, "phone_verified": false"
, "is_account_verified": false"
requires_otp_verification": true"
{

```

ماذا يعني هذا للـ frontend؟

- إذا ضربت أي protected route ورجعت هذه الـ shape، لا تعاملها كخطأ عام فقط.
- حوّل المستخدم إلى شاشة OTP verification.
- هذا مهم خصوصاً في حالات:

- access token قديم
- session قديمة
- refresh/login challenge

11. Decision Matrix لل frontend

الحالة	ماذا تفعل؟
201 -> register ومع requires_otp_verification=true	افتح شاشة OTP ولا تخزن tokens
200 -> login	خزن access_token و refresh_token وانتقل لل app
403 -> login ومع requires_otp_verification=true	افتح شاشة OTP ولا تخزن tokens
200 -> verify-otp	خزن tokens وأغلق شاشة OTP
200 -> refresh	استبدل access_token فقط
403 -> refresh ومع requires_otp_verification=true	افتح شاشة OTP
403 -> protected route ومع requires_otp_verification=true	افتح شاشة OTP
200 -> resend-verification-otp	أعد تشغيل countdown فقط
422 -> verify-otp	اعرض Invalid or expired OTP

12. Recommended frontend state

عند الانتقال إلى شاشة OTP، يفضل أن تخزن state مثل التالي:

```

}
, "flow": "register"
, "registration_method": "phone"
, "email": null
, "phone": "+491234567890"
"masked_destination": "*****7890"
{

```

أو:

```

}
, "flow": "login"
, "registration_method": "email"
, "email": "ahmad@example.com"
, "phone": null
"masked_destination": "ah***@example.com"
{

```

الحقول المقترحة داخل OTP screen state

```

flow: register | login | refresh | protected_route | password_reset •
                                registration_method •
                                email •
                                phone •
                                masked_destination •

```

هذا يسهل عليك:

- تنفيذ verify-otp
- تنفيذ resend-verification-otp
- عرض النص الصحيح للمستخدم

13. Error Handling و Validation

1. أخطاء Laravel validation العادية

هذه عادة تأتي بهذا الشكل:

```

}
, ".message": "The given data was invalid"
} : "errors"
] : "registration_method"
".The selected registration method is invalid"
, [
] : "phone"
".The phone field is required"
[
{
{

```

2. أخطاء OTP invalid/expired

هذه تأتي بشكل مختلف قليلاً:

```

}
"error": "Invalid or expired OTP"
{

```

3. أخطاء refresh token أو credentials

```

}
"message": "Invalid credentials"
{

```

أو:

```

}
"message": "Invalid or expired refresh token"
{

```

توصية مهمة

لا تبنيوا منطق الـ frontend على مقارنة نص message حرفياً.
ابنوا المنطق على:

- status code
- وجود requires_otp_verification

- وجود access_token
- وجود errors
- وجود error

14. أفضل ممارسة للـ frontend

افعلوا هذا

1. وُحِّدُوا شاشة OTP لتخدم:

- بعد register
- بعد login challenge
- بعد refresh challenge
- بعد protected route challenge

2. اجعلوا الـ frontend يرسل فقط identifier المناسب:

- email إذا registration_method=email
- phone إذا registration_method=phone

3. حافظوا على phone format موحد، ويفضل E.164 مثل:

491234567890+

4. خزنوا refresh_token فقط عندما يأتي من:

- login الناجح
- verify-otp الناجح

5. تعاملوا مع 403 + requires_otp_verification= true كحالة flow معروفة وليس ك generic error.

لا تفعلوا هذا

1. لا تتوقعوا tokens من register .
2. لا تعتمدوا على otp داخل الـ response.
3. لا تفترضوا أن email موجود دائماً لكل users.
4. لا تفترضوا أن refresh يعيد refresh_token جديد.
5. لا تربطوا منطق التطبيق بنصوص الرسائل الإنجليزية نفسها.

15. مثال عملي كامل: Email Registration Flow

Step 1: Register

```
}  
  , "first_name": "Ahmad"  
  , "last_name": "Ali"  
  , "registration_method": "email"  
  , "email": "ahmad@example.com"  
  , "password": "Password@123"  
  , "password_confirmation": "Password@123"  
{
```

النتيجة:

- 201
- بدون tokens
- مع requires_otp_verification=true

Step 2: افتح شاشة OTP

اعرض مثلاً:

تم إرسال رمز التحقق إلى ah***@example.com

Step 3: Verify OTP

```
}  
  , "registration_method": "email"  
  , "email": "ahmad@example.com"  
  , "otp": "123456"  
{
```

النتيجة:

- 200
- يوجد access_token
- يوجد refresh_token
- requires_otp_verification=false

Step 4: ادخل التطبيق

- خزّن tokens
- خزّن user data
- انتقل إلى home/dashboard

16. مثال عملي كامل: Unverified Login Flow

Step 1: Login

```
}  
, "registration_method": "phone"  
, "phone": "+491234567890"  
"password": "Password@123"  
{
```

Step 2: إذا رجع 403

مثال:

```
}  
'message': "Your account is not verified. A new OTP has been sent to your registered contact"  
, "registration_method": "phone"  
, "verification_channel": "phone"  
, "masked_destination": "*****7890"  
, "is_account_verified": false  
requires_otp_verification": true  
{
```

Step 3: افتح شاشة OTP مباشرة

Step 4: Verify OTP

```
}  
,"registration_method": "phone"  
,"phone": "+491234567890"  
"otp": "123456"  
{
```

Step 5: خزّن tokens

بعد 200 , خزّن:

- access_token
- refresh_token
- token_type
- user

17. ما الذي يحتاجه فريق الـ frontend لتعديل التطبيق؟

المطلوب تحديثه في التطبيق

1. شاشة التسجيل ترسل registration_method .

2. شاشة login تدعم طريقتين:

- email + password

- phone + password

3. شاشة OTP موحدة وتدعم:

- verify

- resend

4. session manager يتعامل مع:

- login -> 200

- login -> 403 challenge

- refresh -> 200

- refresh -> 403 challenge

5. global API error handler يتعامل مع:

- requires_otp_verification=true + 403

- ويحوّل المستخدم إلى شاشة OTP إذا لزم

أقل تعديل منطقي مقترح في frontend architecture

اجعلوا عندكم auth states مثل:

guest
pending_verification
authenticated

بحيث:

- بعد register : الحالة تصبح pending_verification
- بعد login 403 challenge : الحالة تصبح pending_verification
- بعد verify-otp 200 : الحالة تصبح authenticated
- بعد logout : الحالة تصبح guest

18. الخلاصة النهائية

الـ API الجديدة لم تعد تعتبر register نهاية عملية الدخول، بل بداية flow التنفيع.

الترتيب الصحيح أصبح:

للمساب الجديد

register -> OTP screen -> verify-otp -> store tokens -> enter app

للمساب الموجود وغير المفعل

login -> 403 verification challenge -> OTP screen -> verify-otp -> store tokens -> enter app

للمساب المفعل

login -> 200 -> store tokens -> enter app

إذا التزم الـ frontend بهذا الـ flow فلن يحتاج لأي افتراضات إضافية، وسيكون متوافقاً بالكامل مع الـ backend الحالي.